First-Come,First-Served CPU Scheduling Algorithm

Completed

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>
int main() {
    int n, i;
    printf("Enter number of processes: \n");
    scanf("%d", &n);
    int AT[n], BT[n], CT[n], TAT[n], WT[n];
    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: \n", i + 1);
        scanf("%d %d", &AT[i], &BT[i]);
    }
    CT[0] = AT[0] + BT[0];
    for(i = 1; i < n; i++)
    {
        if(CT[i-1] < AT[i])
            CT[i] = AT[i] + BT[i];
        else
            CT[i] = CT[i-1] + BT[i];
    }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- $Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)$
- $Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)$

Input Format

1. The first line of input contains an integer representing the number of processes

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>
int main() {
    int n, i, completed = 0, time = 0, min_index;
    printf("Enter number of processes: \n");
    scanf("%d", &n);
    int at[n], bt[n], ct[n], tat[n], wt[n], done[n];
    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        done[i] = 0;
    }
    float total_tat = 0, total_wt = 0;
    while(completed < n) {
        min_index = -1;
        for(i = 0; i < n; i++) {
            if(at[i] <= time && done[i] == 0) {
                if(min_index == -1 || bt[i] < bt[min_index]) {
                    min_index = i;
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int n, i, completed = 0, time = 0, min_index;

    printf("Enter number of processes: \n");
    scanf("%d", &n);

    int at[n], bt[n], pr[n], ct[n], tat[n], wt[n], done[n];

    for(i = 0; i < n; i++) {
        printf("Enter AT, BT and Priority for P%d: \n", i+1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        done[i] = 0;
    }

    float total_tat = 0, total_wt = 0;

    while(completed < n) {
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the Time Quantum.
3. The next lines contain two integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>
int main() {
    int n, tq, i, time = 0, completed = 0;
    printf("Enter number of processes: \n");
    scanf("%d", &n);
    printf("Enter Time Quantum: \n");
    scanf("%d", &tq);
    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n];
    for(i = 0; i < n; i++) {
        printf("Enter AT and BT for P%d: \n", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }
    float total_tat = 0, total_wt = 0;
    while(completed < n) {
        int executed = 0;
        for(i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0) {
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1